

LLM 駆動フィジカル AI のリアルタイム性

主要論文 5 本の整理、Xinu メッシュ RTOS への含意、および研究計画

児玉靖司

2026 年 8 月 12 日

Abstract

基盤モデル (VLA: Vision-Language-Action) でロボットを動かそうとすると、**推論が遅すぎて制御周期に間に合わない**という一点に問題が集約される。本稿は、この遅延をどう扱うかについて異なる立場をとる論文 5 本を読み、その解法を「遅延を隠す／遅延に合わせて整合させる／遅延をスケジュールする／遅延の外側で安全を保証する」の 4 系統に整理する。最後に、xinu-mesh-rtos (AIPL 駆動の 12 ノード Xinu メッシュをアーム型ロボット向け RTOS にする計画) の実測値と突き合わせ、どこが既に足りていて、どこが未着手かを述べる。そのうえで、我々の RTOS を論文にするための**4 つの設計案** (うち 1 つは 6 ノードで済む最小構成) と、機械学習をどの計算機で実装するか構成、および各案の研究予算を示す。本稿は 3 部からなる。第I部で文献を整理し、第II部で 4 つの設計案と予算を示し、第III部で推奨案 (案 D、2 年・220 万円) を独立した研究計画として詳述する。

Contents

I	文献レビュー—— LLM 駆動フィジカル AI のリアルタイム性	3
1	なぜこの 5 本か	3
2	AsyncVLA —— 遠隔の大モデルとエッジの小方策を非同期に繋ぐ	3
3	Jetson-PI —— 機上で完結させ、未来を先読みして整合させる	4
4	VLASH —— 実行時刻の状態に条件づけ直す	5
5	TimelyLLM —— 生成を分割し、時間効用で並べ替える	5
6	Safe LLM-Controlled Robots —— 実行前に到達可能集合で検証する	6
7	整理	6
8	xinu-mesh-rtos への含意	6
9	読み残し	7
II	研究構想—— 4 つの設計案と予算	7
10	論文化に向けた 4 つの設計案	8
10.1	案 A: 鮮度を第一級の OS 抽象にする	8
10.2	案 B: ハード締切と時間効用を単一カーネルで共存させる	9
10.3	案 C: 三層の計算配置と、WCET 保証つき機上推論	9
10.4	案 D: 6 ノード最小構成で三者を同時に satisfy する	10
10.5	4 案の対比	11
11	機械学習をどの計算機で実装するか	11
11.1	まず、学習と推論を分けて考える	11
11.2	推論の三層と、それぞれの計算機	11
11.3	学習側の構成	12
11.4	データをどう集めるか	12

12 研究予算	12
12.1 考え方	12
12.2 案 D：6 ノード最小構成（推奨・小規模）	12
12.3 案 A：鮮度を OS 抽象にする（小規模）	13
12.4 案 B：二層スケジューラ（中規模）	13
12.5 案 C：三層配置と WCET 保証つき機上推論（大規模）	13
12.6 段階的に進める場合の推奨	13
III 研究計画——案 D「文脈の齢を渡す実時間 OS」	13
13 学術的な問い	14
13.1 既存研究との決定的な差	14
14 これまでの準備状況	14
15 研究方法（案 D）	15
15.1 発想の転換——上位層の実物は要らない	15
15.2 システム構成	15
15.3 OS 側の実装項目	16
16 学習の設計——アームの動作を何から、どう学ぶか	16
16.1 学習する範囲と、しない範囲	16
16.2 方策の入出力	16
16.3 学習法——特権教師つき模倣学習	16
16.4 なぜ強化学習を使わないか	17
16.5 対照実験の作り方	17
16.6 量子化と実機への載せ方	17
16.7 データ収集と鮮度ラベル	17
17 外部計測の設計——ロジック・アナライザで何を測るか	17
17.1 なぜ外部計測が要るか	17
17.2 測る 4 つの量	18
17.3 なぜオシロスコープではなくロジック・アナライザか	18
17.4 測定側の落とし穴	18
18 評価計画と成功基準	18
18.1 測る量	18
18.2 合格ライン	19
19 年次計画	19
20 想定される困難と対処	19
21 研究経費（案 D の年次内訳）	20
22 期待される成果と波及	20
22.1 学術的な成果	20
22.2 発表計画	20
22.3 後続研究への接続	20

Part I

文献レビュー—— LLM 駆動フィジカル AI のリアルタイム性

1 なぜこの 5 本か

2026 年現在、LLM をロボットに使う研究の主流は、言語モデルに動作計画を出させる初期の形から、視覚・言語・動作を一体で学習する **VLA 基盤モデル**へ移っている。そこで工学的な中心課題になっているのが**推論遅延**である。

数字で押さえておく。実機のサーボ更新は 50–120 Hz、すなわち 1 周期あたり 8–20 ms しかない。一方、80 億パラメータ級の VLA を機上の Jetson Orin で素直に動かすと 1 推論に **400 ms 超**かかり、制御周波数は 1 Hz を割る。**2 桁の開き**があり、これは実装の巧拙では埋まらない。本稿で扱う 5 本は、この開きに対する立場がそれぞれ異なる (表1)。

Table 1: 5 本の立ち位置

論文	時期	遅延への立場
AsyncVLA	2026-02	隠す。重い推論を遠隔に置き、エッジ側の小さな方策が高頻度で動き続ける
Jetson-PI	2026-07	整合させる (機上完結)。推論完了時点の環境を先読みして予測を補正
VLASH	2025-12	整合させる (学習で)。実行時刻の状態に条件づけるよう学習し直す
TimelyLLM	2024-12	スケジュールする。生成を分割し、時間効用でサーバ側を並べ替える
Safe LLM	2025-03	外側で保証する。到達可能性解析で LLM の出力を検証してから実行

読み方の断り。本稿は各論文の本文 (arXiv の HTML 版) から抽出した記述に基づく。PDF 本体を精読したわけではなく、数値は各論文が自己申告したものである。特に第3節の VLASH との比較は Jetson-PI 側の主張であり、競合手法の評価としてそのまま受け取るべきではない。

2 AsyncVLA —— 遠隔の大モデルとエッジの小方策を非同期に繋ぐ

Hirose, Glossop, Shah, Levine. *AsyncVLA: An Asynchronous VLA for Fast and Robust Navigation on the Edge*. arXiv:2602.13476, 2026 年 2 月。

構成

意味的な推論と反射的な実行を**切り離す**。

- **遠隔側**: OmniVLA (82.6 億パラメータ) を RTX 4090 のワークステーションで走らせる。最大 5 Hz。出力は最終層の動作特徴を 4×4096 から 1024 次元へ圧縮した 動作トークン埋め込みであって、動作そのものではない。
- **エッジ側**: ロボット上の Jetson Orin (30 W) で 7600 万パラメータの ViT ベース方策を 8 Hz で回す。遠隔から来た埋め込み、現在の一人称画像 (96×96)、そして古い観測 I_{t-k} の 3 つを食べて、8 ステップ (3 Hz 制御で 2.4 秒ぶん) の動作チャンクを出す。

肝は「古い入力で学習させる」こと

素直に作れば、遠隔から届く意味情報は必ず古い。本手法は学習時に**わざと観測をずらす**。すなわち「VLA 側の観測は長い文脈からランダムに選び、エッジ側は常に最新の観測を見る」。これによりエッジ方策は、古い意味特徴を受け取りながら現在の状態に対する動作を出すよう強制される。

さらに**軌道の重み付け**を行う。現在参照の動作チャンク A_t と遅延参照の A_{t-k} の最終姿勢が閾値以上離れる標本を重く扱う。これは衝突回避や歩行者への譲りといった反射的にふるまうべき場面を選び出す操作にあたる。

結果

Vizbot 実機、WiFi の実測遅延 0.28–6.0 秒という条件で、動的環境（歩行者あり）の位置指定ナビゲーションを評価している（表2）。

Table 2: AsyncVLA の実機結果（位置指定ナビゲーション）

手法	成功率	到達時間 (s)	静的衝突	動的衝突
OmniVLA-edge (1.08 億・機上のみ・6 Hz)	25 %	80.07	1.00	0.50
OmniVLA (82.6 億・遠隔・5 Hz)	45 %	70.73	0.30	1.05
提案手法から E2E 微調整を除いたもの	25 %	82.78	0.60	1.05
AsyncVLA	85 %	59.18	0.10	0.10

45 %→85 %、すなわち**40 ポイント**の改善である（論文の要旨は「40 % higher」と書くが、表を見るかぎり 40 ポイントが正確）。人為的に 0.2–5.0 秒の遅延を課しても成功率 50 % を保ち、同条件の OmniVLA は 0.2 Hz で 0 % まで落ちる。

示唆。大きいモデルを機上に載せる必要はない。ただし**エッジ側の方策は「古い情報が来る」前提で学習されていなければならない**。これは通信や OS の工夫では埋められない、学習側の要件である。

3 Jetson-PI ——機上で完結させ、未来を先読みして整合させる

Jetson-PI: Towards Onboard Real-Time Robot Control via Foresight-Aligned Asynchronous Inference. arXiv:2607.12659, 2026 年 7 月。

問題の切り分け

機上 VLA には 2 つの別々の問題があるとする。

1. **知覚と実行のずれ**：Jetson Orin では推論に約 1.4 秒かかり、その間に環境が変わる。予測された動作は古い状態に向いている。
2. **反応時間**：非同期化しても、応答は推論遅延 Δ から $\Delta + L$ (L はチャック長) ステップの間に律速される。

手法

VLM の最終層表現の $t + \Delta$ 時点の値を予測する小さな補正モジュール（4000 万パラメータ、全体の 1 %）を置く。画像や層ごとの KV キャッシュではなく、圧縮済みの最終層出力だけを使う。出力は 2 つで、未来の環境表現と確信度である。

この確信度で**VLM を呼ぶかどうかを切り替える**。確信度が閾値を超えていれば VLM を飛ばし、動作エキスパートだけを回す。下回れば VLM に現環境を見せて文脈を更新する。根拠として、機上デバイスでは VLM も動作エキスパートも帯域律速になるため、ハイエンド GPU と違って並列化が逆効果になる、というルーフライン解析を挙げている。実装面では CUDA グラフの再利用、GPU 常駐バッファ、フローマッチングの展開融合を併用する。

結果

- Jetson Orin での制御周波数：素の PyTorch 0.70 Hz → **6.06 Hz** (8.66 倍)。Jetson Thor では 7.59 Hz。
- 内訳(Orin, $\pi_{0.5}$)：ViT 79.5 ms + LLM 210.3 ms + 動作エキスパート 123.1 ms = 合計 **412.9 ms**。反応時間 165.1 ms。
- 実機（布たたみ）：たたみ 8/10 成功。同じ Orin 上の素朴な非同期実装は 0/10。

なお 6.06 Hz は、確信度による VLM 省略を含めた制御周波数であり、VLM を毎回呼ぶ場合の 1 推論は上記の 412.9 ms を要する。

内訳の 412.9 ms は本稿でもっとも重要な数字である。**最適化を尽くしても、機上の VLA は 1 周期あたり数百 ms を要する。**

4 VLASH —— 実行時刻の状態に条件づけ直す

VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference. arXiv:2512.01031, 2025 年 12 月。

同じ「ずれ」に、**学習だけで**対処する。追加モジュールもアーキテクチャ変更もない。

着想は単純である。新しいチャンクが用意できるまで、ロボットは前のチャンクの残りを実行する。その動作は既知なのだから、それに沿って状態を前送りすれば実行時刻の状態を推定できる。そこでモデルを、現在状態ではなく**推定した実行時刻の状態**に条件づける。

学習は時間オフセット拡張で行う。視覚観測は固定したまま状態だけ時間的にずらした組を与え、モデルが視覚特徴に過適合せず状態を見るよう仕向ける。複数オフセットを観測共有で 1 系列に詰める実装により学習が 3.26 倍速い。

結果は、同期推論比で反応遅延が最大 11.8 倍短縮、動作量子化と併せてタスク完了が 1.5–2.0 倍速。卓球（命中率 55%）やもぐら叩きのような動的タスクが成立する。 $\pi_{0.5}$ 、GR00T N1.6、SmolVLA にそのまま適用できる。

Jetson-PI との関係。両者は同じ問題を、追加モジュールで補正する（Jetson-PI）か学習で吸収する（VLASH）かで分かれる。Jetson-PI は LIBERO で VLASH に平均 14.8 ポイント勝ち、遅延 $\Delta = 9$ で VLASH が 30.1% に落ちるのに対し 97.0% を保つと報告する。ただしこれは**Jetson-PI 側による競合手法の評価**である。

5 TimelyLLM —— 生成を分割し、時間効用で並べ替える

Ling, Chen, Zhong. *TimelyLLM: Segmented LLM Serving System for Time-sensitive Robotic Applications.* arXiv:2412.18695, 2024 年 12 月。

唯一の**OS / システムソフトウェア側**の論文であり、本計画にもっとも直接効く。

問題

既存の LLM サービング系は先着順（FCFS）のバッチ処理をとる。これはロボットの時間制約と噛み合わない。ロボットは計画の生成とその実行を交互に行うが、FCFS では応答が全部揃うまで実行を開始できない。

分割生成

生成を最後まで待たず、**実行可能な単位**が出た時点で切り出す。生成中に逐次デトークナイズし、`pick(cyan_box)` のような実行可能文が完成したらそこで生成を中断してロボットへ投げる。再開のために KV キャッシュとトークン列を保持する。この**文脈切り替えのコストは約 9.5 ms**で、典型的な生成時間 370 ms の 3% 未満である。

ロボットが投げた区間を実行している間に次の区間を生成できるため、生成と実行が重なる。

時間効用によるスケジューリング

各要求に時間効用関数 $TUF(t) = \min(\beta, \alpha(t - ERT) + \beta)$ を与える。通常タスクは $ERT = 1s$ 、 $\alpha = -2$ 、緊急タスクは $ERT = 200ms$ 、 $\alpha = -6.67$ とする。スケジューラは**潜在効用密度**（単位時間あたり効用）を貪欲に最大化し、ロボットの実行所要時間の見積りを取り込んで、現在の実行が終わる前に次の区間の生成が始まるよう並べる。

結果

- 評価対象：DJI Tello ドローン（11 タスク）、Neuromeka Indy7 Pro アーム（積み上げ・機の片づけ・分類）、チャットボット。RTX 4090 + Ryzen 9 7950x、Llama3-8B (fp16)。
- アーム（機の片づけ）：時間効用最大 **1.97 倍**、ロボットの待ち時間 **84 % 削減**。
- スケジューラ単体：緊急タスクの効用が FCFS 比 183 %、EDF 比 142 %。
- 分割生成のオーバーヘッドは 6 % 未満、スケジューリングは待ち 8 件で 2.03 ms。

示唆。EDF に 142 % の差をつけている点が重要である。**締切だけを見る古典的スケジューリングでは足りない**。LLM の応答は「遅れると価値が連続的に落ちる」性質を持ち、その形 (α 、ERT) をスケジューラに教える必要がある。

6 Safe LLM-Controlled Robots ——実行前に到達可能集合で検証する

Safe LLM-Controlled Robots with Formal Guarantees via Reachability Analysis. arXiv:2503.03911, 2025 年 3 月。

LLM は零ショットで人と対話できるが、**形式的な安全保証を持たない**。本手法は解析モデルに頼らず、**運用データから**ロボット + LLM 系の到達可能集合を構成する。LLM が出した低レベル制御指令について、起こりうる軌道すべてが安全領域に留まるかを実行前に検証する。自律ナビゲーションとタスク計画の事例で有効性を示している。

これは前 4 本と層が違う。前 4 本が速さの話であるのに対し、本論文は速さを諦めてよい層に安全論証を置く。

7 整理

Table 3: 4 系統の対比

系統	代表	要点と代償
遅延を隠す	AsyncVLA	大モデルは遠隔でよい。代償：エッジ方策を「古い入力」で学習し直す必要。通信断への備えも要る
遅延に整合させる	Jetson-PI / VLASH	機上完結できる。代償：それでも数 Hz。Jetson-PI は追加モジュール、VLASH は再学習が要る
遅延をスケジュールする	TimelyLLM	生成と実行を重ねる。代償：分割点を「実行可能単位」で切れる出力形式が前提
外側で保証する	Safe LLM	形式保証が得られる。代償：検証のぶん遅くなる。速い層には置けない

共通しているのは、**単一の周期に統合しようとしていない**ことである。5 本とも、速い層と遅い層を分け、その間を古い情報でも動くように設計している。

8 xinu-mesh-rtos への含意

既に足りているもの

本計画では rpi5 について、preempt=1 で 1 kHz 周期タスクの **最大ジッタ 3 μ s**、**200 周期中の締切超過 0**を実測している (rpi4 は 228 μ s、rpi3 は 57 μ s)。

これを分野の要求と並べる (表4)。

System 1 の時間予算は、Xinu メッシュなら 3 桁の余裕がある。8.3 ms に対してジッタ 3 μ s である。つまり本計画がこれまで積み上げてきた RT 化 (優先度付き ready_pop、tickless ワンショットタイマ、ネットスタックの ISR からの追い出し) は、分野が必要としている速い層の要件を既に満たしている。

Table 4: 要求周期と実測値の対比

層	周期	1 周期の予算
System 1 (サーボ指令)	120 Hz	8.3 ms
両腕系のループ全体	50 Hz	20 ms
System 2 (VLM 文脈)	5–10 Hz	100–200 ms
機上 VLA の 1 推論 (Jetson-PI, $\pi_{0.5}$, Orin)	—	412.9 ms
同・最適化と VLM 省略を入れた制御周波数	6.06 Hz	—
本計画の実測 (rpi5, 1 kHz)	1000 Hz	ジッタ 3 μ s

未着手なもの

不足しているのは速さではなく、**遅い層との繋ぎ方**である。具体的には 3 点。

1. **System 2 をどこで走らせるか**。412.9 ms を要する推論を 12 ノードのどこに置くか、あるいは AsyncVLA のように外部に置くかは未決である。Xinu ノードに載る規模ではない以上、**メッシュの外に置いて非同期に受ける**のが現実的だと思われる。
2. **古い文脈を配る仕組み**。5–10 Hz で届く意味トークンを、1 kHz で回る 12 ノードへどう配布し、各ノードがどれだけ古い情報を持っているかをどう表現するか。AIPL のアクター・メッセージングはこのチャンネルの実装として素直だが、現状の実装は**アクターのメールボックスが溢れると落とす** (cc.c:532)。古い意味トークンを落とすのは正しい挙動でありうるが、意図してそうしている状態にはなっていない。
3. **時間効用の考え方**。TimelyLLM が EDF に 142% の差をつけた事実は、本計画のスケジューラ設計に直接効く。速い層は締切で正しいが、**遅い層は締切ではなく効用で並べるべき**である。現在の優先度固定スケジューリングには、この二層構造がない。

さらに、学習側の要件が OS 設計を縛る

AsyncVLA のもっとも重い含意は性能値ではない。**エッジ側の方策は「古い意味情報がある」前提で学習されていなければ、どれだけ OS を速くしても成立しない**という点である。逆に言えば、本計画が提供すべきは遅延ゼロではなく、**遅延が有界であること、そしてその値が上位に見えること**である。「いま持っている文脈は何 ms 前のものか」をノードが答えられる設計にしておけば、上位の学習側はそれを条件として扱える。これは RTOS 側が出せる保証として自然であり、いまの実装には無い。

9 読み残し

本稿は arXiv の HTML 版から抽出した記述に基づく整理であり、以下は未確認である。

- AsyncVLA の所属機関、および言語指定ナビゲーションの詳細な条件。
- Jetson-PI の VLASH 比較 (LIBERO) の再現条件。競合手法の評価は提案側の報告のみで判断すべきでない。
- Safe LLM 論文の検証にかかる計算コスト。実時間制御の内側に置けるかは不明。
- いずれの論文も分散構成 (複数ノードで 1 台のロボットを制御) は扱っていない。12 ノード構成に固有の課題は、この 5 本の外にある。

Part II

研究構想—— 4 つの設計案と予算

10 論文化に向けた 4 つの設計案

第8節で述べたとおり、速い層は既に足りている。新規性を出すべきなのは**OS 側で何を保証するか**であって、モデルを速くすることではない。既読 5 本との差分は次の 3 点に集約される。

- いずれも**分散構成を扱っていない**。1 台の計算機（あるいは遠隔 1 台＋機上 1 台）が前提。
- 遅延への対処が**学習側に寄っている**。OS が何を保証するかは書かれていない。
- 機上推論の**WCET（最悪実行時間）を論じていない**。平均の Hz は出すが上界は出さない。

以下の案 A-C はこの 3 点にそれぞれ対応し、案 D は 3 点すべてを最小構成で同時に狙う。

10.1 案 A：鮮度を第一級の OS 抽象にする

主張

「この文脈は何 μ s 前のものか」を、OS が保証し、かつ上位に見せる。

AsyncVLA はエッジ方策を「古い情報が来る」前提で学習して解いた。本案は同じ問題を OS の契約として解く。各アクター・メッセージに生成時刻を持たせ、配送時に実測経過時間を添える。さらに経路ごとの**鮮度上界 D** を静的に与え、 D を超えた文脈は配送せずに破棄する（現状の cc.c:532 の暗黙の取りこぼしを、明示的な鮮度契約に置き換える）。

実装項目

1. AIPL のメッセージ型に鮮度上界を付与し、アクター合成の経路について D を**静的に**積み上げる型検査（rtos_plan の課題⑩の具体化）。
2. 実行時 API `ctx_age_us()`：いま保持している文脈の実測齢を返す。
3. メールボックス方針の明示化：文脈クラスは最新優先で上書き、指令クラスは取りこぼし禁止。クラスごとに別方針を持たせる。
4. 12 ノードでの端点間鮮度の実測。**平均ではなく分布**で示す（上界の主張は最悪値でしか支えられない）。

評価と主張できること

静的上界 D と実測最大値の**乖離（tightness）**が主要指標になる。加えて、同一方策に対して鮮度を条件として与えた場合と与えない場合で制御性能を比較すれば、「OS が鮮度を見せることの効用」を定量化できる。これは既読 5 本のどれも測っていない量である。

規模とリスク

最小構成なら 3 ノードで論文になる。実装は既存カーネルの延長で、新規ハードウェアをほぼ必要としない。**弱点は上位の学習側**で、鮮度条件つき方策を自前で用意できないと「見せた効用」を示せない。簡易な模倣学習方策で代替する退路を用意しておく。

投稿先

RTSS / RTAS、あるいは ACM TECS。言語側を前面に出すなら EMSOFT。

10.2 案 B：ハード締切と時間効用を単一カーネルで共存させる

主張

1kHz の関節制御（締切で正しい層）と、5–10 Hz の文脈配布（効用で正しい層）を、同じベアメタル・カーネルの上で隔離しながら同時に最適化する。

TimelyLLM は EDF に 142 % の差をつけたが、あれはサーバ側 GPU のスケジューリングである。ロボット側で、しかも硬いリアルタイム制御と同居させた例はない。本案は二層スケジューラを作る。

- ・ **クラス 1（硬）**：周期タスク。固定優先度 + tickless ワンショット。既に rpi5 で 1 kHz・ジッタ 3 μ s・0 ミスを実測済み。**これを一切劣化させない**のが制約。
- ・ **クラス 2（軟）**：文脈配布・AIPL アクター。時間効用関数 $TUF(t) = \min(\beta, \alpha(t - ERT) + \beta)$ で並べ替え、クラス 1 の空き時間だけを使う。

実装項目

1. 二層 ready キューと、クラス 2 がクラス 1 を妨げないことの**実測による隔離検証**（クラス 2 を過負荷にしてもクラス 1 が 0 ミスを保つこと）。
2. 効用密度による貪欲スケジューラ。TimelyLLM の PUD をベアメタルへ移植し、ロボット側の実行時間見積りを組み込む。
3. 分割配布：10 Hz で届く意味トークンを関節ごとの断片に切り、効用順に 12 ノードへ配る。TimelyLLM の分割生成の配送側にあたる。
4. GC の分離（rtos_plan 課題③）。クラス 2 に AIPL の GC を閉じ込める。

評価と主張できること

比較対象は FCFS・EDF・固定優先度。主張は 2 つ、**(i) クラス 1 の 0 ミスを保ったまま、(ii) クラス 2 の効用が EDF を上回る**。TimelyLLM の 142 % がベアメタル・分散でどう変わるかを示せば、それ自体が独立した知見になる。

規模とリスク

12 ノードが要る。**最大のリスクは時刻同期**で、効用の比較にはノード間で共通の時間軸が必要になる。PTP 対応スイッチを予算に入れる。

投稿先

RTSS が本命。DATE / EMSOFT も射程。

10.3 案 C：三層の計算配置と、WCET 保証つき機上推論

主張

ニューラルネットの推論そのものを、ベアメタル上で WCET 上界つきで回す。そのうえで、12 関節 × 異種 3 世代ボードへの配置を実機フィットネスで合成する。

既読 5 本は平均の制御周波数（6.06 Hz など）は出すが、**上界を出していない**。GPU とランタイム（CUDA、動的メモリ、キャッシュ）を前提にする限り、上界は原理的に出しにくい。ベアメタル Xinu なら、静的確保・固定小数点・キャッシュ制御込みで上界が言える。これは本計画にしか主張できない立ち位置である。

既存の評価との関係

本計画の 2026 年 7 月 21 日の評価（6 軸アームの制御に *Xinu* メッシュは効くか）は、「**関節サーボループにメッシュは効果がなく、むしろ有害**」と結論している。本案はこれと矛盾しない。**1 関節 1 ノードとし、制御ループは各ノード内に閉じる**からである。ネットワークを跨ぐのは 5–10 Hz の意味トークンだけであり、1 kHz のループがネットワークを待つ構成は採らない。同評価が「効く」とした動作計画・分散知覚・冗長性が、本案では第 2 層・第 3 層にあたる。

三層配置（詳細は第11節）

- 第 3 層（意味）：VLA 基盤モデル、5–10 Hz、メッシュ外の GPU。
- 第 2 層（動作）：中規模方策（～7600 万パラメータ）、8–30 Hz、Jetson。
- 第 1 層（関節）：**小型量子化方策（≤100 万パラメータ）＋インピーダンス制御、1 kHz、各 *Xinu* ノード上**。ここに WCET 保証をかける。

実装項目

- NEON int8 の固定小数点推論カーネル（GEMM）を、動的確保なし・分岐なしで実装し、**サイクル単位の上界**を与える。
- D-cache の明示管理（既に 3 ボードで成立済みの資産をここへ効かせる）。
- 12 関節 ×3 世代への配置合成。*xinu-mesh-ga* の実機フィットネス方式を流用する。既に**sim-to-real gap**を定量化した実績がある（モデル予測 1775 ms に対し実測 2050 ms、手作業の均等分割が 1427 ms で最速）。同じ現象が RT 配置でも起きるかを問う。
- 混在クリティカリティ：関節ごとに重要度が違う（把持は安全上重い）ことを配置の制約として扱う。

評価と主張できること

WCET 上界と実測分布の乖離、および**異種世代の非自明な順位**。既に「分岐の多い探索では A53 (rpi3) が A72 (rpi4) に勝つ」という反直感的な実測がある。NN 推論でも同種の逆転が起きるなら、モデルベースの配置設計が誤ることの証拠になる。

規模とリスク

3 案でもっとも大きい。実機アームと 36 ボード規模が要る。**最大のリスクは第 1 層方策の表現力**で、100 万パラメータで関節制御が成り立つかは未検証である。まず 1 関節・シミュレーションで成立を確認してから実機へ広げる段取りとする。

投稿先

RTAS / EMSOFT、journal なら IEEE TCAD や ACM TECS。機械学習側の会議（CoRL など）には向かない。貢献が OS 側にあるためである。

10.4 案 D：6 ノード最小構成で三者を同時に satisfy する

案 A–C はいずれも百万円単位の機材を前提にする。案 D は**6 ノードと小型アーム 1 台**に絞り、それでも機械学習・フィジカル AI・分散実時間 OS の三者が交わる主張を単独で完結させる。

主張はこうである——**文脈の齢を入力に持つ方策は、齢を持たない同一構造の方策より、遅延が変動する条件下で追従誤差が小さい。そしてその齢を信頼できる形で供給できるのは OS だけである。**

実行可能性を支えるのは一つの発想の転換である。**遅延の研究に、遅延を生む実物は必要ない。**VLA は購入も実行もせず、上位層を既読論文の実測分布に従う遅延源として再現する。これで機材費の大半が消え、しかも遅れ方を制御変数として掃引できる。

本案は本稿の推奨であり、第III部で独立した研究計画として詳述する。以下ではまず 4 案を横に並べる。

10.5 4 案の対比

Table 5: 3 案の対比

	新規性の核	必要規模	直接経費
案 D 最小	齢を情報として方策に渡す。三者の交点を最小構成で	6 ノード+小型アーム	2,200 千円 (2 年)
案 A 鮮度	鮮度を OS の契約にし、上位へ見せる	3 ノード+計測器	4,200 千円 (3 年)
案 B 二層	硬締切と時間効用の単一カーネル共存	12 ノード+同期網	9,600 千円 (3 年)
案 C 三層	ベアメタル NN 推論の WCET 上界+実機配置合成	36 ボード+実機アーム	14,500 千円 (3 年)

推奨は案 D から入ることである。案 D は既存 3 ボードに 3 枚を買い足すだけで組め、しかも機械学習・フィジカル AI・分散実時間 OS の三者が交わる主張を単独で完結させられる。ここで得た鮮度の実測分布と、齢が効きはじめる境界が、そのまま案 A・案 B の入力になる。案 C は案 B の成果（時刻同期・配置基盤）が前提になるため、単独で先に始めると重い。

11 機械学習をどの計算機で実装するか

11.1 まず、学習と推論を分けて考える

研究室規模で**VLA 基盤モデルを一から学習するのは不可能**である。80 億パラメータ級の事前学習には数百 GPU 日を要し、既読論文の多くも公開済みモデル（ $\pi_{0.5}$ 、GR00T、OmniVLA、SmolVLA）を出発点にしている。

したがって設計方針は決まる。**大きいモデルは凍結して使い、小さいアダプタだけを学習する**。これは AsyncVLA の構成そのものであり、同時に予算的にも実行可能な唯一の道である。学習対象は次の 2 つに限る。

- 第 2 層の**エッジ・アダプタ**（～7600 万パラメータ）。単一 GPU で微調整可能。
- 第 1 層の**関節方策**（≤100 万パラメータ）。GPU 1 枚で数時間規模。

11.2 推論の三層と、それぞれの計算機

Table 6: 推論の三層配置

層	規模	周期	計算機	根拠
第 3 層意味	10^9	5–10 Hz	GPU ワークステーション（メッシュ外）	機上に載る規模ではない。AsyncVLA の遠隔配置に倣う
第 2 層動作	10^7 – 10^8	8–30 Hz	Jetson AGX Orin / Thor（境界ノード 1 台）	Jetson-PI の実測で 6–7.6 Hz。ViT 級はここが下限
第 1 層関節	≤ 10^6	1 kHz	各 Xinu ノード（A76, NEON int8）	下記の見積り。ここだけが WCET を言える層

第 1 層が Xinu ノードで成立するかの見積り

100 万パラメータの全結合方策を 1 kHz で回すと、 $2 \times 10^6 \text{ FLOP} \times 10^3 = 2 \text{ GFLOP/s}$ である。Cortex-A76 の NEON (128 bit、int8) なら演算能力としては十分な余裕がある。一方で第 2 層の

7600 万パラメータを同じ場所で回すと 1.2 GFLOP/s 相当だが、こちらは**帯域律速**になり、しかもベアメタルには BLAS も NPU もない。

したがって**第 1 層は Xinu ノード上で成立しうるが、第 2 層は成立しない**。この線引きが案 C の出発点である。なお Jetson-PI が指摘した「機上では VLM も動作エキスパートも帯域律速になり、並列化が逆効果になる」という観察は、NPU を持たない Xinu ノードでは一層強く効く。

既存資産との接続

mecharm 系で構想している「12 Xinu + TinyML 強化学習」は、まさにこの第 1 層にあたる。本案はそれを WCET 保証つきにする点で前進させる。

11.3 学習側の構成

- 学習機：RTX 5090 (32 GB) 級を 2 枚、CPU は多コア、主記憶 256 GB、NVMe 8 TB。7600 万パラメータのアダプタ微調整と、時間オフセット拡張 (VLASH 方式) による再学習がここに収まる。
- 推論機(第 3 層)：上記の学習機を兼用する。実験時はロボットと同一 LAN に置く。AsyncVLA の実測 (WiFi 0.28–6.0 秒) を見るかぎり、有線にすれば遅延は大幅に小さくなるはずで、**その差自体が案 A の鮮度分布の材料**になる。
- 機上 (第 2 層)：Jetson AGX Orin 64 GB を 1 台。Jetson-PI が同系で 6.06 Hz を出しているため、比較の基準機として妥当。
- 保管：実機ログは鮮度ラベル付きで貯める。40 TB 級の NAS。

11.4 データをどう集めるか

学習データは**買えない**。実機から採るしかない。

1. 遠隔操作による模倣データ：リーダー・アーム型の操作装置で人が動かし、軌道を記録する。数百エピソード規模。
2. 鮮度ラベル：本計画に固有の要件である。全記録に**そのとき文脈が何 μ s 古かったか**を付ける。これが無いと、案 A の「鮮度を条件として与える」学習ができない。通常データ収集系には無い項目なので、収集系の設計時に入れておく必要がある。
3. シミュレーション：第 1 層方策の事前学習に使う。既存の DOFBOT 忠実シミュレータを流用できる。

12 研究予算

12.1 考え方

3 案とも 3 年計画とし、直接経費で示す (間接経費 30 % は別途)。以下は概算であり、機材価格は 2026 年 8 月時点の想定である。

計測器を必ず計上すること。OS が自分でジッタを測ると循環論法になる。本計画がこれまで得た「1kHz でジッタ 3 μ s」も、カーネル内部の計時に依存している。**査読で必ず突かれる**ので、ロジック・アナライザで GPIO を外から観測した独立な裏付けを用意する。これは 3 案すべてに共通する必須項目である。

12.2 案 D：6 ノード最小構成 (推奨・小規模)

2 年・直接経費 2,200 千円 (内訳と年次配分は第 III 部第 21 節)。科研費なら**挑戦的研究 (萌芽)**あるいは**基盤研究 (C)**の小さめの枠、学内研究費でも射程に入る。旅費と人件費を別財源で賄えるなら物品費 870 千円で着手できる。

12.3 案 A：鮮度を OS 抽象にする（小規模）

Table 7: 案 A の予算（3 年・直接経費、単位：千円）

費目	内訳	金額
物品費	ロジック・アナライザ（16ch, 500 MS/s 級）	400
	追加ノード（Raspberry Pi 5 ×4、電源・SD・配線）	200
	学習・推論用ワークステーション（RTX 5090 ×1、256 GB）	900
	測定用オシロスコープ（4ch, 200 MHz）	300
旅費	国際会議（RTSS / EMSOFT）2 回	900
	国内研究会 3 回	200
人件費・謝金	研究補助（学生 RA）3 年	600
その他	英文校閲、投稿料・OA 費用	550
	消耗品（基板、ケーブル、記録媒体）	150
合計		4,200

科研費**基盤研究 (C)**（3 年・総額 500 万円以内）の枠に収まる。新規ハードウェアが少なく、既存の 3 ボードで着手できるのが利点である。

12.4 案 B：二層スケジューラ（中規模）

Table 8: 案 B の予算（3 年・直接経費、単位：千円）

費目	内訳	金額
物品費	12 ノード分のボード（Pi5 ×8、Pi4 ×4）と電源・筐体	800
	PTP 対応スイッチ、配線、時刻同期用機材	400
	ロジック・アナライザ、オシロスコープ	700
	学習・推論用ワークステーション（RTX 5090 ×2、256 GB、8 TB）	1,800
	Jetson AGX Orin 64 GB（第 2 層の基準機）	400
	6 軸アーム実機（評価用）	700
旅費	国際会議 4 回（うち 2 回は学生同行）	1,800
	国内	300
人件費・謝金	学生 RA 2 名 ×3 年	1,800
その他	英文校閲、投稿料・OA、消耗品	900
合計		9,600

科研費**基盤研究 (B)**相当。案 A の成果を前提にすると採択の説得力が上がる。

12.5 案 C：三層配置と WCET 保証つき機上推論（大規模）

科研費**基盤研究 (B) の上限～(A)**相当。実機アームを扱うため**安全対策の費用を必ず計上する**こと（囲い、非常停止、リスクアセスメント）。フィジカル AI は人を傷つけ得るという本計画自身の問題意識と整合する。

12.6 段階的に進める場合の推奨

総額を一度に取りにいくより、**案 D で「齢を渡す」という主張を先に論文化しておく**方が、後続の申請で「我々が提唱した枠組み」と書けるぶん有利になると思われる。案 D は 2 年・220 万円で、しかも**三者（機械学習・フィジカル AI・分散実時間 OS）が交わる主張を単独で完結させられる**点が要である。

Table 9: 案 C の予算 (3 年・直接経費、単位：千円)

費目	内訳	金額
物品費	実機アーム (6 軸 ×2 台、または双腕 1 台)	2,500
	遠隔操作装置 (リーダー・アーム、データ収集用)	300
	ボード 36 枚 (3 世代 ×12)、電源・ラック・冷却	1,600
	PTP 網、計測器一式 (ロジアナ・オシロ・電力計)	1,100
	学習用ワークステーション (RTX 5090 ×2) + Jetson AGX Thor	2,300
	データ保管 NAS 40 TB	400
旅費	国際会議 5 回、国内	2,600
人件費・謝金	学生 RA 3 名 ×3 年	2,700
その他	英文校閲、投稿料・OA、消耗品、安全対策 (囲い・非常停止)	1,000
合計		14,500

Table 10: 推奨する進め方

時期	案	狙い
1-2 年目	案 D (萌芽／基盤 C, 220 万円)	6 ノード＋実機アームで着手。齢が効く境界を同定し、最初の論文にする
2-5 年目	案 A (基盤 C, 420 万円)	案 D の分布を根拠に、鮮度を型と契約へ一般化する
5-8 年目	案 B (基盤 B, 960 万円)	12 ノードへ拡大。時刻同期基盤を整える
8 年目-	案 C (基盤 B～A, 1,450 万円)	案 B の基盤の上で WCET 保証つき機上推論へ

Part III

研究計画——案 D 「文脈の齢を渡す実時間 OS」

13 学術的な問い

本研究が立てる問いは一つである。

文脈の「齢」を情報として与えられた制御方策は、与えられない方策より良く振る舞うか。振る舞うとすれば、どの条件下でか。そして、その齢を信頼できる形で供給できるのは誰か。

最後の問いに対する本研究の答えは**オペレーティングシステムである**。アプリケーションは自分がいつ止められ、いつ再開されたかを知らない。経過時間を正しく測れるのは、止めた側だけである。

13.1 既存研究との決定的な差

AsyncVLA は、エッジ方策を「古い意味情報が来る」前提で学習することで解いた。学習時にわざと観測をずらし、古い特徴から現在の動作を出すよう強制する。これは巧妙だが、**方策に齢そのものを渡してはいない**。渡せなかったのである—— OS が教えないからである。

本研究はその一手先に行く。齢を暗黙に吸収させるのではなく、**明示的な入力として渡す**。すると方策は、「文脈が新しいときは意味情報を信じ、古いときは自分の状態推定を重んじる」といった条件分岐を学習できる。

14 これまでの準備状況

本計画は白紙から始まるものではない。

Table 11: 既に到達している地点

資産	状態
ベアメタル RTOS	Embedded Xinu を Raspberry Pi 3/4/5 (A53/A72/A76) へ移植済。優先度付きプリエンプティブ・スケジューラ、tickless ワンショット・タイマ、ネットワークスタックのタイマ ISR からの分離を実装済
実時間性の実測	rpi5 で 1kHz 周期タスクのジッタ 3μs・200 周期中の締切超過 0 (全コア負荷下)。rpi4 は 228 μ s、rpi3 は 57 μ s
アクター言語 AIPL	自作。型付きメッセージ交換による並行記述。3 ボード上で動作
分散実行基盤	4 コア SMP $\times$ 3 ボード、Wi-Fi アドホック・メッシュ、12 コア分散実行を実証済
アーム制御の設計	6 軸アーム DOFBOT を 1 軸 1 ノードで制御する全体設計を文書化済 (2026 年 7 月、9 ページ)。忠実な 3D シミュレータも実装済
HIL の実績	ドローン救助シミュレーションと実機 Xinu のハードウェア・イン・ザ・ループを構築済

すなわち**速い層は既に足りている**。System 1 (サーボ指令) の要求が 120 Hz = 1 周期 8.3 ms であるのに対し、実測はジッタ 3 μ s である。**3 桁の余裕**がある。不足しているのは速さではなく、**遅い層との繋ぎ方**である。

15 研究方法 (案 D)

15.1 発想の転換——上位層の実物は要らない

本研究の実行可能性を支える鍵はここにある。

遅延の研究に、遅延を生む実物は必要ない。リアルタイム系の研究では、負荷を負荷モデルで置き換えるのが標準的な作法である。

したがって本研究は VLA 基盤モデルを**購入せず、動作させない**。代わりに上位層を**実測分布に従う遅延源**として再現する。分布は既読論文が公開している値 (AsyncVLA の WiFi 実測 0.28–6.0 秒、Jetson-PI の 1 推論 412.9 ms) を用い、必要なら手元の 1 台で一度だけ較正する。

これには二つの利点がある。第一に**機材費の大半が消える**。第二に、そしてより重要なことに、**遅れ方を制御変数として掃引できる**。実物を使うと遅延は与件になるが、モデルなら独立変数にできる。本研究の主眼は「どの遅延条件下で齢が効きはじめるか」の同定であるから、掃引できることは実物に対する優位性である。

15.2 システム構成

Table 12: 6 ノードの構成

要素	台数	役割
関節ノード	6	6 軸アームの各関節に 1 台。1 kHz の制御ループを 自ノード内で閉じて 回す。小型量子化方策 (≤ 5 万パラメータ、int8) をベアメタルで実行
上位層代理	1	実測分布に従って遅れる目標姿勢・意味トークンを 5–10 Hz で配る。関節ノードの 1 台に相乗りさせるか、手元 PC に置く
被制御対象	1	6 軸小型アーム実機 (DOFBOT 級)
外部計測	1 式	ロジック・アナライザ。GPIO を外から観測し、 カーネル内部の計時に依存しない裏付け を取る

既存の 3 ボード (rpi3/rpi4/rpi5) に 3 枚を買い足せば 6 ノードになる。**異種 3 世代が混ざる構成**は偶然の産物だが、そのまま「混在ハードウェアにおける鮮度上界」という副次的な問いを生む。同一構成 6 台より論点が多い。

15.3 OS 側の実装項目

1. **齢の測定と伝播**。各メッセージに生成時刻を持たせ、配送時に実測経過時間を添える。実行時 API `ctx_age_us()` を提供する。
2. **鮮度契約**。経路ごとに上界 D を与え、 D を超えた文脈は配送せず破棄する。現在の実装はメールボックス溢れ時に暗黙に取りこぼしている (`cc.c:532`)。これを意図された方針に置き換える——文脈クラスは最新優先で上書き、指令クラスは取りこぼし禁止とする。
3. **ノード間の時刻同期**。齢を比較するには共通の時間軸が要る。6 ノード規模なら簡素な同期で足りる見込みだが、同期誤差が齢の測定誤差に直接効くため、同期誤差自体を測って報告する。
4. **ベアメタル推論**。NEON int8 の固定小数点カーネルを、動的確保なし・データ依存分岐なしで実装し、WCET 上界を与える。

16 学習の設計——アームの動作を何から、どう学ぶか

16.1 学習する範囲と、しない範囲

まず範囲をはっきりさせる。**アームが「何をするか」は本計画では学習しない**。それは上位層 (VLA) の仕事であり、本計画ではそれを遅延源として模擬する (第10.4節)。学習するのは下の層である。

遅れて届いた指令を、その遅れ具合を知ったうえで、どう実行するか。

タスク水準の行動学習 (把持の巧拙、物体の選択など) は案 C の範囲であり、本計画はそこへ至る前提にあたる時間的整合の問題だけを扱う。範囲を狭めることが、2 年・220 万円という規模を成立させている。

16.2 方策の入出力

1 関節あたり 1 つの方策を置く。

Table 13: 関節方策の入出力 (1 関節あたり)

内容	次元
入力 自関節の角度 θ 、角速度 $\dot{\theta}$ 、追従誤差、積分項	4
上位から来た目標 θ_{target} 、 $\dot{\theta}_{\text{target}}$	2
齢 a (μs、正規化)	1
隣接関節の状態 (メッシュ経由。これも齢つき)	4-6
出力 関節指令 (位置または電流)	1

12 次元前後の入力、隠れ層 64×2 、1 出力で**5 千–1 万パラメータ**である。上限に置いた 5 万には余裕がある。畳み込みも注意機構も要らない。**1 kHz で回すには、この規模でなければ入らない** (第11節の見積り)。

16.3 学習法——特権教師つき模倣学習

設計の肝はここにある。

1. **教師を作る** (シミュレータ上)。既存の DOFBOT 忠実シミュレータで、**遅延ゼロ**の理想コントローラ (軌道生成 + PD / インピーダンス制御) を走らせる。教師は「いま本当の目標がどこにあるか」を知っている。
2. **生徒には遅れた情報だけ渡す**。同じ場面で、目標を人工的に遅らせたものとその齢だけを生徒に見せ、教師と同じ指令を出すよう学習させる。

3. **遅延を掃引してデータを作る**。0.1 ms から数百 ms まで遅延を変えた軌跡を生成する。VLASH の時間オフセット拡張と同じ発想である。
4. **実機で微調整**。摩擦・バックラッシュ・サーボの応答遅れはシミュレータと合わない。遠隔操作で数十エピソードを集め、出力層近くだけを調整する。

教師は未来を知っており、生徒は知らない。しかし生徒は**自分がどれだけ遅れているかは知っている**。この非対称性が学習の駆動力になる。ロボティクスでは確立した枠組み (privileged learning / Learning by Cheating 系) であり、**手法自体の新規性を主張する必要がない**。そのぶん主張を「齢の有無」の一点に絞れるのが利点である。

16.4 なぜ強化学習を使わないか

6 軸アームで 1 kHz の強化学習は、実機では危険であり、データ効率も悪すぎる。加えて本研究の問いは「最適制御の発見」ではなく**齢の情報が効くか**である。教師との差がそのまま評価量になる模倣学習のほうが、問いに直結する。

ただし限界を明記しておく。教師自身が失敗する大遅延領域は、模倣では学べない。遅延が十分大きければ遅延ゼロの教師に追随することは原理的に不可能であり、その領域は本計画の範囲外とする。「どこまで教師に追随でき、どこから追随できなくなるか」の境界を示すことが、本計画が約束できる範囲である。

16.5 対照実験の作り方

- 方策 P0 (齢なし) : 入力から a を除く。
- 方策 P1 (齢あり) : a を含む。

この 2 つを**同一構造・同一データ・同一学習量**で学習させる。P0 にも a の入力端子を持たせて常にゼロを流すなど、**容量を厳密に揃える**こと。揃えないと、測っているのは齢の効果ではなく単なる容量差になる。**実験計画で最も間違えやすい点**である。

16.6 量子化と実機への載せ方

学習は float32 で行い、事後量子化で int8 に落とす。Xinu ノード上では固定小数点で推論し、動的確保もデータ依存分岐も持たない (第10.4節の WCET 要件)。**量子化誤差が制御性能をどれだけ損なうかも測る**。ベアメタルでの量子化推論と制御品質の関係は、それ自体が報告に値する。

16.7 データ収集と鮮度ラベル

学習データは買えない。実機とシミュレータから採る。ここで**通常の収集系にはない項目**が要る。

全記録に「そのとき文脈が何 μs 古かったか」を付す。

これが無ければ、齢を条件として与える学習そのものがない。収集系の設計時点に入れておく必要があり、後から足すことはできない (過去の記録に遡って齢を復元する手段がないため)。

17 外部計測の設計——ロジック・アナライザで何を測るか

17.1 なぜ外部計測が要るか

OS が自分でジッタを測ると循環論法になる。研究代表者が既に得ている「1 kHz でジッタ 3 μs 」も、カーネル内部の計時に依存しており、外からの裏付けを持たない。本計画の主張は「**OS が齢を保証する**」であるから、その保証自体を外部から検証しないかぎり、主張は自己言及で閉じてしまう。

Table 14: ロジック・アナライザで測る量

量	測り方	内部計測で可能か
①周期タスクの起動時刻	RT タスクの先頭で GPIO を 1 本トグルし、立ち上がりエッジの時刻列を記録。 間隔の分布がそのまま真のジッタ になる	可能だが要裏付け
②割り込み応答遅延	外部から既知のタイミングでパルスを入力し、応じる ISR が別ピンを立てるまでを測る。入力エッジ → 出力エッジの時間	不可能
③端点間の齢	ノード A (上位層代理) が送出する瞬間と、ノード B (関節) が制御ループで消費する瞬間に、それぞれピンを立てる。 2 信号の時間差が真の齢	不可能
④推論の実行時間／ノード間の時刻同期誤差	推論カーネルの入口・出口をトグルし、パルス幅の最大値を取る。同期プロトコル上の同一時刻に全ノードがピンを立て、そのズレを見る	不可能 (同期誤差)

17.2 測る 4 つの量

③が本計画の土台である。外部で測った真の齢と、ノードが `ctx_age_us()` で自己申告した値を突き合わせ、**OS の申告が正しいことを外から示す**。これができて初めて、「齢を渡す」という主張が成立する。

17.3 なぜオシロスコープではなくロジック・アナライザか

多チャンネルを共通の時間軸で同時記録できるからである。③のようにノードをまたぐ時間差を測るには、1 台の計器で全ノードの信号を同時に見なければならない。6 ノード × 2 信号 + 上位層 + トリガで 14 チャンネル程度、16 チャンネル機で足りる。

加えて、裾 (99.9 パーセントイル) を論じるには $1\text{kHz} \times 10\text{秒} = 1\text{万周期規模}$ の連続記録が要る。生のサンプルで保持するのは非現実的なので、**遷移のみを圧縮記録できる機種**を選ぶ必要がある。分解能は 2–10 ns あれば足りる (3 μs のジッタに対して 3 桁の余裕)。

17.4 測定側の落とし穴

- GPIO トグル自体のコスト**が測定値に乗る (数十–数百 ns)。空ループでのトグル間隔を較正して差し引く。
- 観測が対象を変える**。GPIO 書き込みはバスとキャッシュに影響する。**トグルあり／なしで制御性能に差が出ないことを確認し**、影響が無視できる範囲であることを示す。
- 計器自身のサンプリング・ジッタ (± 1 サンプル) と、配線長による伝搬遅延 (数 ns)。いずれも本計画の μs 水準では無視できるが、無視できると書くために一度は測る。

18 評価計画と成功基準

18.1 測る量

いずれも第17節の外部計測で取る。

- 端点間の齢の分布**。平均ではなく最悪値と裾で示す。6 ノード分を重ねる。
- 齢あり／齢なしの追従誤差**を、上位層の遅延分布を掃引しながら比較する。

3. **WCET 上界と実測分布の乖離**（上界の tightness）。異種 3 世代で別々に出す。
4. **対照実験**：齢を意図的に誤って渡した場合の劣化。OS の保証が効いていることは、保証を壊してみせて初めて示せる。

18.2 合格ライン

Table 15: 定量的な成功基準

項目	基準
6 ノード同時稼働時の関節制御 端点間の齢 齢の効果	1 kHz を維持し、締切超過 0 99.9 パーセントイルが設計上界 D 以内 遅延の標準偏差が制御周期の一定倍を超える領域で、追従誤差 RMS を 有意に改善 （改善幅そのものは事前に約束しない）
WCET の tightness 外部計測との整合	実測最大が上界の 80 % 以上（緩すぎる上界は役に立たない） ロジック・アナライザによる実測とカーネル内計時の差が既知の範囲

「**齢の効果**」の基準を数値で約束していないのは意図的である。効果の大きさは遅延分布に依存し、事前には決まらない。本研究が約束するのは効果の有無と、効きは始める境界の同定である。

19 年次計画

Table 16: 2 年間の計画

時期	主題	内容
1 年目前期	基盤整備	6 ノード化。齢の測定・伝播 API と鮮度契約の実装。外部計測系（ロジック・アナライザ）の確立と、既存の 1 kHz/3 μ s 実測の外部からの再確認
1 年目後期	学習と模擬	小型方策のシミュレータ上での学習。齢あり／なしの比較をまずシミュレーションで実施。遅延負荷モデルの較正
2 年目前期	実機統合	DOFBOT 実機と 6 ノードの統合。ベアメタル推論の WCET 解析。端点間の齢の実測分布を取得
2 年目後期	掃引と執筆	遅延掃引実験、対照実験。国内研究会で発表し、査読つき論文へ投稿

1 年目後期の時点で、**シミュレーションだけで主張の成否が判る**ように組んである。そこで効果が見えなければ、2 年目の実機統合の前に問いの立て方を修正できる。

20 想定される困難と対処

1. **齢を渡しても差が出ない**。最大のリスクである。遅延が小さい領域では齢の情報は役に立たないはずで、効くのは遅延が制御周期に対して大きい領域に限られる。**これは失敗ではなく境界の同定である**。「どこから齢が効きは始めるか」を示せればそれ自体が結果になるよう、評価計画を掃引型にしてある。
2. **小型方策の表現力が足りない**。5 万パラメータで関節制御が成り立つかは未検証である。まず 1 関節・シミュレーションで成立を確認してから 6 軸へ広げる段取りとする。成立しない場合は、方策を補正項に限定し、主たる制御は古典的なインピーダンス制御に委ねる構成へ退く。
3. **時刻同期誤差が齢の測定精度を支配する**。同期誤差そのものを測って報告し、齢の議論をその精度の範囲に限定する。必要なら有線化する。
4. **実機アームの分解能が足りず、制御誤差の差が観測できない**。DOFBOT 級の安価なアームはエンコーダ分解能が低い。シミュレーションでの結果を主、実機を「実在することの証拠」と位置づける二段構えにしておく。

21 研究経費（案 D の年次内訳）

Table 17: 2 年間の直接経費（単位：千円）

費目	内訳	1 年目	2 年目	計
物品費	6 軸小型アーム実機（DOFBOT 級）1 台	—	120	120
	Raspberry Pi 5 ×3 追加、電源・配線・筐体	100	—	100
	ロジック・アナライザ（16ch）	400	—	400
	学習用 GPU 1 枚（既存 PC への増設）	250	—	250
旅費	国際会議 1 回（WiP / ポスター想定）	—	450	450
	国内研究会 3 回	100	100	200
人件費・謝金	学生 RA	200	200	400
その他	英文校閲、投稿料・OA、消耗品	80	200	280
合計		1,130	1,070	2,200

計測器を必ず計上している点を強調しておく。OS が自分でジッタを測ると循環論法になる。研究代表者が既に得ている「1 kHz でジッタ 3 μs」もカーネル内部の計時に依存しており、**査読で必ず問われる**。外部からの独立な裏付けは省略できない。

さらに切り詰める場合。旅費と人件費を別財源で賄えるなら、物品費 870 千円で着手できる。ただし削ってよいものとそうでないものがある。アームを数値積分（HIL）に置き換えれば 120 千円減るが、**実機で動かした図のない論文はフィジカル AI としては弱い**。ロジック・アナライザは学内の共用設備で代替しうるが、**計測そのものを省いてはならない**。

22 期待される成果と波及

22.1 学術的な成果

1. 「**齢を渡す**」という新しい設計軸の提示と、その有効領域の同定。
2. ベアメタル上のニューラルネット推論に対する **WCET 上界**の実測つき提示。既存研究は平均の制御周波数しか示していない。
3. 異種世代混在環境における鮮度上界の実測。

22.2 発表計画

まず国内（情報処理学会組込みシステム研究会／システムソフトウェアとオペレーティング・システム研究会）で議論を経たうえで、査読つき国際会議（RTAS）または論文誌（ACM TECS）へ投稿する。規模の小ささを踏まえ、国際会議では Work-in-Progress 枠も現実的な選択肢とする。

22.3 後続研究への接続

本計画で得る **齢の実測分布と、齢が効きはじめる境界**は、そのまま次の段階の入力になる。

- 鮮度を AIPL の型として表現し、経路ごとの上界を静的に検査する（言語設計への展開）。
- ハード締切クラスと時間効用クラスを単一カーネルで共存させるスケジューラ設計（12 ノードへの拡大）。

すなわち本計画は、**最小の投資で主張の核を先に確立し、後続の大型計画で「我々が提唱した枠組み」と書けるようにする**ことを狙っている。

文献

- [1] N. Hirose, C. Glossop, D. Shah, S. Levine. *AsyncVLA: An Asynchronous VLA for Fast and Robust Navigation on the Edge*. arXiv:2602.13476, 2026. <https://arxiv.org/abs/2602.13476>
- [2] *Jetson-PI: Towards Onboard Real-Time Robot Control via Foresight-Aligned Asynchronous Inference*. arXiv:2607.12659, 2026. <https://arxiv.org/abs/2607.12659>
- [3] *VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference*. arXiv:2512.01031, 2025. <https://arxiv.org/abs/2512.01031>
- [4] N. Ling, G. Chen, L. Zhong. *TimelyLLM: Segmented LLM Serving System for Time-sensitive Robotic Applications*. arXiv:2412.18695, 2024. <https://arxiv.org/abs/2412.18695>
- [5] *Safe LLM-Controlled Robots with Formal Guarantees via Reachability Analysis*. arXiv:2503.03911, 2025. <https://arxiv.org/abs/2503.03911>